

Research Notes on Neural architecture search

Neural architecture search

>> Section 1

Bayesian optimization Bayesian Optimization (BO), which has proven to be an efficient method for hyperparameter optimization, can also be applied to NAS. In this context, the objective function maps an architecture to its validation error after being trained for a number of epochs. At each iteration, BO uses a surrogate to model this objective function based on previously obtained architectures and their validation errors. One then chooses the next architecture to evaluate by maximizing an acquisition function, such as expected improvement, which provides a balance between exploration and exploitation. Acquisition function maximization and objective function evaluation are often computationally expensive for NAS, and make the application of BO challenging in this context. Recently, BANANAS has achieved promising results in this direction by introducing a high-performing instantiation of BO coupled to a neural predictor.

>> Section 2

Neural architecture search benchmarks Neural architecture search often requires large computational resources, due to its expensive training and evaluation phases. This further leads to a large carbon footprint required for the evaluation of these methods. To overcome this limitation, NAS benchmarks have been introduced, from which one can either query or predict the final performance of neural architectures in seconds. A NAS benchmark is defined as a dataset with a fixed train-test split, a search space, and a fixed training pipeline (hyperparameters). There are primarily two types of NAS benchmarks: a surrogate NAS benchmark and a tabular NAS benchmark. A surrogate benchmark uses a surrogate model (e.g.: a neural network) to predict the performance of an architecture from the search space. On the other hand, a tabular benchmark queries the actual performance of an architecture trained up to convergence. Both of these benchmarks are queryable and can be used to efficiently simulate many NAS algorithms using only a CPU to query the benchmark instead of training an architecture from scratch.

>> Section 3

Reinforcement learning Reinforcement learning (RL) can underpin a NAS search strategy. Barret Zoph and Quoc Viet Le applied NAS with RL targeting the CIFAR-10 dataset and achieved a network architecture that rivals the best manually-designed architecture for accuracy, with an error rate of 3.65, 0.09 percent better and 1.05x faster than a related hand-designed model. On the Penn Treebank dataset, that model composed a recurrent cell that outperforms LSTM, reaching a test set perplexity of 62.4,

or 3.6 perplexity better than the prior leading system. On the PTB character language modeling task it achieved bits per character of 1.214. Learning a model architecture directly on a large dataset can be a lengthy process. NASNet addressed this issue by transferring a building block designed for a small dataset to a larger dataset. The design was constrained to use two types of convolutional cells to return feature maps that serve two main functions when convoluting an input feature map: normal cells that return maps of the same extent (height and width) and reduction cells in which the returned feature map height and width is reduced by a factor of two. For the reduction cell, the initial operation applied to the cell's inputs uses a stride of two (to reduce the height and width). The learned aspect of the design included elements such as which lower layer(s) each higher layer took as input, the transformations applied at that layer and to merge multiple outputs at each layer. In the studied example, the best convolutional layer (or "cell") was designed for the CIFAR-10 dataset and then applied to the ImageNet dataset by stacking copies of this cell, each with its own parameters. The approach yielded accuracy of 82.7% top-1 and 96.2% top-5. This exceeded the best human-invented architectures at a cost of 9 billion fewer FLOPS—a reduction of 28%. The system continued to exceed the manually-designed alternative at varying computation levels. The image features learned from image classification can be transferred to other computer vision problems. E.g., for object detection, the learned cells integrated with the Faster-RCNN framework improved performance by 4.0% on the COCO dataset. In the so-called Efficient Neural Architecture Search (ENAS), a controller discovers architectures by learning to search for an optimal subgraph within a large graph. The controller is trained with policy gradient to select a subgraph that maximizes the validation set's expected reward. The model corresponding to the subgraph is trained to minimize a canonical cross entropy loss. Multiple child models share parameters, ENAS requires fewer GPU-hours than other approaches and 1000-fold less than "standard" NAS. On CIFAR-10, the ENAS design achieved a test error of 2.89%, comparable to NASNet. On Penn Treebank, the ENAS design reached test perplexity of 55.8.

>> Section 4

Hill-climbing Another group used a hill climbing procedure that applies network morphisms, followed by short cosine-annealing optimization runs. The approach yielded competitive results, requiring resources on the same order of magnitude as training a single network. E.g., on CIFAR-10, the method designed and trained a network with an error rate below 5% in 12 hours on a single GPU.

Research Notes on Neural architecture search

Neural architecture search

>> Section 5

The search space defines the type(s) of ANN that can be designed and optimized. The search strategy defines the approach used to explore the search space. The performance estimation strategy evaluates the performance of a possible ANN from its design (without constructing and training it). NAS is closely related to hyperparameter optimization and meta-learning and is a subfield of automated machine learning (AutoML).

>> Section 6

Neural architecture search (NAS) is a technique for automating the design of artificial neural networks (ANN), a widely used model in the field of machine learning. NAS has been used to design networks that are on par with or outperform hand-designed architectures. Methods for NAS can be categorized according to the search space, search strategy and performance estimation strategy used:

>> Section 7

One-shot models RL or evolution-based NAS require thousands of GPU-days of searching/training to achieve state-of-the-art computer vision results as described in the NASNet, mNASNet and MobileNetV3 papers. To reduce computational cost, many recent NAS methods rely on the weight-sharing idea. In this approach, a single overparameterized supernet (also known as the one-shot model) is defined. A supernet is a very large Directed Acyclic Graph (DAG) whose subgraphs are different candidate neural networks. Thus, in a supernet, the weights are shared among a large number of different sub-architectures that have edges in common, each of which is considered as a path within the supernet. The essential idea is to train one supernet that spans many options for the final design rather than generating and training thousands of networks independently. In addition to the learned parameters, a set of architecture parameters are learnt to depict preference for one module over another. Such methods reduce the required computational resources to only a few GPU days. More recent works further combine this weight-sharing paradigm, with a continuous relaxation of the search space, which enables the use of gradient-based optimization methods. These approaches are generally referred to as differentiable NAS and have proven very efficient in exploring the search space of neural architectures. One of the most popular algorithms amongst the gradient-based methods for NAS is DARTS. However, DARTS faces problems such as performance collapse due to an inevitable aggregation of skip connections and poor generalization which were tackled by many future algorithms. Methods like aim at robustifying DARTS and making the validation accuracy landscape smoother by

introducing a Hessian norm based regularisation and random smoothing/adversarial attack respectively. The cause of performance degradation is later analyzed from the architecture selection aspect. Differentiable NAS has shown to produce competitive results using a fraction of the search-time required by RL-based search methods. For example, FBNet (which is short for Facebook Berkeley Network) demonstrated that supernet-based search produces networks that outperform the speed-accuracy tradeoff curve of mNASNet and MobileNetV2 on the ImageNet image-classification dataset. FBNet accomplishes this using over 400x less search time than was used for mNASNet. Further, SqueezeNAS demonstrated that supernet-based NAS produces neural networks that outperform the speed-accuracy tradeoff curve of MobileNetV3 on the Cityscapes semantic segmentation dataset, and SqueezeNAS uses over 100x less search time than was used in the MobileNetV3 authors' RL-based search.